

AI Brain — implementation vs theory

May 2026

English

Here is a clear decoder of the AI Brain section, followed by a file-by-file gap analysis you can defend in front of the jury.

1 — What the document says, in clear words

The section describes the AI brain as a system with one orchestrator + six specialised agents + a safety guard, all on Azure AI Foundry.

Role	What it does	Why it is built this way
Topic Router	Reads the utterance, detects the language, keeps conversation state in Redis, picks the right agent to call	Latency-critical, high-volume, low-token — a small fast model is enough
Request Classifier	Adds a tag: intent / agency / language / urgency	Pre-sorting before the heavier agents fire
Translator	Translates across the 12 languages while preserving the administrative jargon	Azure AI Translator for plain prose, reasoning model for sensitive passages
Document Extractor	Reads passports, payslips, leases and returns structured fields, redacting PII the downstream agent does not need	Azure AI Document Intelligence does the OCR, the model structures the result
Citizen Assistant	Answers questions, must cite a docId from the knowledge base	APIM blocks the response if the citation is missing
Caseworker Helper	Helps the caseworker: draft reply, summary, next-best-action	Purely advisory, never operative
Eligibility Pre-Assessor	Assesses eligibility, proposes a verdict and the evidence	The only agent classified high-risk per AI Act Annex III §5(b)

The non-negotiable rule: Eligibility never makes a final decision. It proposes. A human caseworker arbitrates. And this verdict-proposal is tamper-evident (hash in Confidential Ledger) so a regulator can replay it 6 months later.

The three “models” mentioned (frontier reasoning, low-latency routing, real-time speech) are Azure OpenAI deployment aliases whose targets are **gpt-5.4**, **gpt-5.4-mini**, **gpt-realtime**. The tenant ↔ alias binding is driven by a Bicep parameter, therefore replaceable.

2 — The implementation vs theory gap, file by file

Claim of the document	What actually exists in the repo	Honest status
7 Foundry agents with managed identity, Entra-only, risk class	<code>foundry/agents/{topic-router,classifier,translator,doc-extractor,citizen-assistant,caseworker-helper,eligibility}/agent.yaml</code> — all present with model, tools, content-safety, risk level	Contracts implemented. Foundry runtime deployment depends on the tenant. Implemented.
AI Act registry per agent Topic Router → Redis for slot-filling	7 files in <code>governance/ai-act/registry/*.yaml</code> + a script <code>Validate-AIRegistry.ps1</code> <code>infra/data/redis/redis-enterprise.bicep</code> provisions Redis. The tools <code>read_session_redis/write_session_redis</code> are declared in <code>agent.yaml</code> but their Foundry runtime implementation is opaque (Foundry calls them for us)	Implemented. Infra present, contract declared, runtime opaque. For the demo: Redis exists, slot-filling works as long as the tenant has a valid gpt-5.4 deployment.
APIM /agent-topic-router/messages Voice → <code>lookup_topic_router</code> function tool	<code>services/apim/apis/agent-topic-router/openapi.yaml</code> + <code>policy.xml</code> <code>apps/voice/call-automation/src/foundry-tool.ts</code> + Bicep <code>voice-orchestrator.bicep</code> + <code>gpt-realtime-deployment.bicep</code>	Live demonstrator. Live demonstrator (subject to gpt-realtime in the region).
Eligibility in Confidential Container SEV-SNP	<code>infra/security/confidential-compute/eligibility-confidential-app.bicep</code> exists but the default image is <code>mcr.microsoft.com/azuredocs/containerapps-helloworld:latest</code> and the workload profile is <code>Consumption</code> (not a confidential SKU D4adsv5/D8adsv5)	Bicep skeleton in place. Image and confidential compute profile not yet wired. Today, Eligibility inference runs in the standard Foundry hub. Blueprint.
Verdict hashed into Azure Confidential Ledger Champion-challenger: 5 % shadow, gold eval, KS test, bias 30 d, alias flip Norway → Sweden for gpt-realtime (sovereignty exception)	<code>infra/security/confidential-ledger/confidential-ledger.bicep</code> provisions the resource. The <code>lineage-writer</code> tool is declared on the agent side but the code that computes the hash and POSTs to the ledger is not in the repo No CI script, no GitHub Action, no job implementing these gates. Mentions only in <code>docs/biz/ai.md</code> , <code>docs/tech/architecture.md</code> , and the presentation <code>voice-orchestrator.bicep</code> accepts <code>country</code> and <code>location</code> . <code>gpt-realtime-deployment.bicep</code> can target another region. The "single Bicep flip" bypass is technically possible but the runtime wiring (NO orchestrator calling SE endpoint) remains to be validated end-to-end	Resource provisionable. Anchoring pipeline = blueprint. Design pattern documented, not implemented in CI. Blueprint. Infra ready, end-to-end not yet exercised on the tenant.

~~infra/foundry/deployments~~ The folder ~~infra/foundry/~~ does not exist in the repo. The two Foundry-runtime Bicep files exist under `apps/voice/call-automation/infra/gpt-realtime-deployment.bicep`

Imprecise reference in the PDF — the mentioned file does not exist under that path.

3 — How to explain it to the jury

You can say, in a single breath:

The repo contains the contracts: 7 agent.yaml files, 7 AI Act registry entries, the APIM OpenAPI for the Topic Router, the Container App Bicep for the Eligibility TEE, the Confidential Ledger Bicep, and the voice orchestrator with its function tool for gpt-realtime. These 7 contracts are a concrete delivery, not wishful thinking.

Live today: Topic Router, Translator, Document Extractor and Citizen Assistant in Demos 1 to 4. The voice channel in Demo 2 uses the real function tool. The Caseworker Helper is implemented agent-side but driven by the strangler-fig pending the D365 licences.

Blueprint that I own: the Eligibility TEE — I have the Container App, I have the Confidential Ledger, I do not yet have the confidential image nor the lineage-writer pipeline. The champion-challenger lifecycle — the pattern is documented but no CI job runs it today. This is precisely what Gates 1 and 2 of the roadmap close.

Three short sentences to hold in reserve if pressed:

1. On the models: *"The aliases `gpt-5.4`, `gpt-5.4-mini`, `gpt-realtime` are targets. On a tenant that does not have them yet, the Bicep parameter `modelDeploymentName` routes to `gpt-4o-mini` without application change. The quality regression would be visible in the gold-set; today the gold-set is not a CI job, it is a YAML definition."*
2. On the Eligibility TEE: *"Bicep ready, standard image. The switch to a SEV-SNP confidential image is a two-parameter change: `workloadProfileName` to `Confidential` and `image` to the signed container. The Foundry-side code does not change — that is the point of the pattern."*
3. On the NO sovereignty exception: *"The Norwegian voice orchestrator already talks to a parameterised endpoint. Today on the demonstrator, lacking `gpt-realtime` in norwayeast on the tenant side, I can point it at `swedencentral` with a single parameter flip. Compliance still holds: transcripts persisted in NO, inference in SE under EU Data Boundary."*

This gap is exactly what the *production-oriented demonstrator* status authorises — you ship all the contracts, demos 1-4 and 10 are live, and you have an explicit roadmap (Gates 1-4) to close the rest.

Français

Voilà un décodeur clair de la section AI Brain, suivi d'un gap analysis fichier-par-fichier que tu pourras défendre devant le jury.

1 — Ce que dit le document, en clair

La section décrit le cerveau IA comme un système à un orchestrateur + six agents spécialisés + une garde sécurité, le tout posé sur Azure AI Foundry.

Rôle	Ce que ça fait	Pourquoi c'est fait comme ça
Topic Router	Lit l'utterance, détecte la langue, garde l'état de la conversation dans Redis, choisit le bon agent à appeler	Latence faible, gros volume, peu de tokens → un "petit" modèle suffit
Request Classifier	Met une étiquette : intention / agence / langue / urgence	Pré-tri avant les agents lourds
Translator	Traduit en 12 langues en préservant le jargon administratif	Service Azure AI Translator pour la phrase de base + un modèle de raisonnement pour les passages sensibles
Document Extractor	Lit passeports/payslips/baux et renvoie des champs structurés, en redactant le PII inutile	Azure AI Document Intelligence fait l'OCR, le modèle structure
Citizen Assistant	Répond aux questions, oblige à citer un docId de la base de connaissances	APIM bloque la réponse si la citation manque
Caseworker Helper	Aide le caseworker : brouillon de réponse, résumé, next-best-action	Purement consultatif, jamais décisionnel
Eligibility Pre-Assessor	Évalue l'éligibilité d'une demande, propose un verdict + l'évidence	Le seul agent classé high-risk au sens AI Act Annex III §5(b)

La règle de fer : Eligibility ne prend jamais une décision finale. Il propose. Un human caseworker arbitre. Et ce verdict-proposition est tamper-evident (hash dans Confidential Ledger) pour qu'un régulateur puisse le rejouer 6 mois plus tard.

Les trois "modèles" mentionnés (frontier reasoning, low-latency routing, real-time speech) sont des alias de déploiement Azure OpenAI dont la cible est `gpt-5.4`, `gpt-5.4-mini`, `gpt-realtime`. Le binding tenant ↔ alias est piloté par un Bicep param, donc remplaçable.

2 — Le gap implémentation / théorie, fichier par fichier

Claim du document	Ce qui existe vraiment dans le repo	Statut honnête
7 agents Foundry avec managed identity, Entra-only, risk class	<code>foundry/agents/{topic-router,classifrier,translator,doc-extractor,citizen-assistant,caseworker-helper,eligibility}/agent.yaml</code> — tous présents avec model, tools, content-safety, risk level	Contracts implémentés. Le déploiement Foundry runtime dépend du tenant.
Registre AI Act par agent	7 fichiers dans <code>governance/ai-act/registry/*.yaml</code> + un script <code>Validate-AIRegistry.ps1</code>	Implémenté.

Claim du document	Ce qui existe vraiment dans le repo	Statut honnête
Topic Router → Redis pour slot-filling	<code>infra/data/redis/redis-enterprise.bicep</code> provisionne Redis. Les outils <code>read_session_redis/write_session_redis</code> sont déclarés dans l' <code>agent.yaml</code> mais leur implémentation Foundry runtime est opaque (Foundry l'appelle pour nous)	Infra présente, contrat déclaré, runtime opaque. Pour la démo : Redis existe, slot-filling marche tant que le tenant a un déploiement gpt-5.4 valide. Live demonstrator.
APIM /agent-topic-router/messages Voice → <code>lookup_topic_router</code> function tool	<code>services/apim/apis/agent-topic-router/openapi.yaml</code> + <code>policy.xml</code> <code>apps/voice/call-automation/src/foundry-tool.ts</code> + Bicep <code>voice-orchestrator.bicep</code> + <code>gpt-realtime-deployment.bicep</code>	Live demonstrator (sous réserve de gpt-realtime dans la région).
Eligibility dans Confidential Container SEV-SNP	<code>infra/security/confidential-compute/eligibility-confidential-app.bicep</code> existe mais l'image par défaut est <code>mcr.microsoft.com/azuredocs/containerapps-helloworld:latest</code> et le workload profile est <code>Consumption</code> (pas un SKU confidential D4adsv5/D8adsv5)	Squelette Bicep en place. Image et profil de calcul confidentiel pas encore branchés. Aujourd'hui, l'inference Eligibility tourne dans le hub Foundry standard. Blueprint.
Verdict hashé dans Azure Confidential Ledger Champion-challenger : 5 % shadow, gold eval, KS test, bias 30 j, alias flip Norvège → Suède pour gpt-realtime (sovereignty exception)	<code>infra/security/confidential-ledger/confidential-ledger.bicep</code> provisionne la ressource. L'outil <code>lineage-writer</code> est déclaré côté agent mais le code qui calcule le hash et POST sur le ledger n'est pas dans le repo Aucun script CI, aucune GitHub Action, aucun job qui exécute ces gates. Mentions uniquement dans <code>docs/biz/ai.md</code> , <code>docs/tech/architecture.md</code> , et la présentation	Ressource provisionnable. Pipeline d'ancrage = blueprint. Design pattern documenté, non implémenté en CI. Blueprint.
<code>infra/foundry/deployments</code> (déploiements modèles)	<code>voice-orchestrator.bicep</code> accepte <code>country</code> et <code>location</code> . <code>gpt-realtime-deployment.bicep</code> peut viser une autre région. La bascule "un seul flip Bicep" est techniquement possible mais le câblage runtime (orchestrateur NO qui appelle endpoint SE) reste à valider en bout-en-bout <code>infra/foundry/deployments/infra/foundry/</code> n'existe pas dans le repo. Les deux Bicep liés à Foundry runtime existent dans <code>apps/voice/call-automation/infra/gpt-realtime-deployment.bicep</code>	Infra prête, end-to-end pas encore exercé sur le tenant. Référence imprécise dans le PDF — le fichier mentionné n'existe pas sous ce chemin.

3 — Comment l'expliquer au jury

Tu peux dire, en une seule respiration :

Le repo contient les contrats : 7 fichiers `agent.yaml`, 7 entrées AI Act registry, l'OpenAPI APIM du Topic Router, le Container App Bicep pour la TEE Eligibility, le Confidential Ledger Bicep, et le voice orchestrator

avec son function tool pour gpt-realtime. Ces 7 contrats sont une livraison concrète, pas du wishful thinking.

Ce qui est live aujourd'hui : Topic Router, Translator, Document Extractor et Citizen Assistant dans les Démon 1-4. Le voice channel Démon 2 utilise le vrai function tool. Le caseworker Helper est implémenté côté agent mais piloté par le strangler-fig en attendant les licences D365.

Ce qui est blueprint et que j'assume : la TEE Eligibility — j'ai le Container App, j'ai le Confidential Ledger, je n'ai pas encore l'image confidentielle ni le pipeline lineage-writer. Le champion-challenger lifecycle — le pattern est documenté mais aucun job CI ne l'exécute aujourd'hui. C'est précisément ce que ferment les Gates 1 et 2 de la roadmap.

Trois phrases courtes à mettre en réserve si on te creuse :

1. Sur les modèles : *"Les alias gpt-5.4, gpt-5.4-mini, gpt-realtime sont des cibles. Sur un tenant qui ne les a pas encore, le param Bicep modelDeploymentName route vers gpt-4o-mini sans changement applicatif. La régression qualité serait visible dans la gold-set ; aujourd'hui la gold-set n'est pas un job CI, c'est une définition YAML."*
2. Sur la TEE Eligibility : *"Bicep prêt, image standard. La bascule vers une image SEV-SNP confidentielle se fait en changeant deux params : workloadProfileName vers Confidential et image vers le container signé. Le code Foundry-side ne change pas — c'est l'intérêt du pattern."*
3. Sur la sovereignty exception NO : *"Le voice orchestrator norvégien parle déjà à un endpoint paramétré. Aujourd'hui sur le démonstrateur, faute de gpt-realtime en norwayeast côté tenant, je peux le pointer vers sweden-central en un flip de param. La conformité reste OK : transcripts persistés en NO, inférence en SE sous EU Data Boundary."*

Ce gap, c'est exactement ce que t'autorise le statut *production-oriented demonstrator* — tu livres tous les contrats, les démos 1-4 et 10 sont live, et tu as une roadmap explicite (Gate 1-4) pour fermer le reste.